

JENKINS-03

Task	Repository	Flow
Task 1	VProfile-1.git	Git Clone → SonarQube → Slack
Task 2	hiring-app.git	Git Clone → SonarQube → Slack

1. Environment & SonarQube Setup

1. Install / Verify Docker and Pull SonarQube Image

Navigation / Tabs	Action Performed	Result / Evidence
EC2 Terminal → yum → Docker	1. Run `yum install docker -y`. 2. Run `docker --version`. 3. Run `docker pull sonarqube`. 4. Run `docker images` to verify the image.	`sonarqube:latest` is available and Docker is ready.

```
[root@ip-172-31-7-96 /]# yum install docker -y
Last metadata expiration check: 1 day, 1:43:43 ago on Thu Aug 13 10:38:39 2026.
Package docker-25.0.16-1.amzn2023.0.3.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-7-96 /]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-7-96 /]# docker pull sonarqube
Using default tag: latest
latest: Pulling from library/sonarqube
966c395d29cb: Pull complete
e114ca80d2ed: Pull complete
f6c4211cf8b1: Pull complete
4f4fb700ef54: Pull complete
b73294fa41f7: Pull complete
7d7d329955b7: Pull complete
2a4d1635d2a3: Pull complete
Digest: sha256:9dfc19385a3b7647e7585013d8c94ae2322228c6b217alc641199f07f86b12ab
Status: Downloaded newer image for sonarqube:latest
docker.io/library/sonarqube:latest
[root@ip-172-31-7-96 /]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
sonarqube     latest   b82115d34675   8 days ago    1.43GB
[root@ip-172-31-7-96 /]# docker run
"docker run" requires at least 1 argument.
See 'docker run --help'.

Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

2. Start SonarQube Container

Navigation / Tabs	Action Performed	Result / Evidence
EC2 Terminal → Docker	1. Run the corrected `docker run` command. 2. Map host port `9000` to container port `9000`. 3. Run `docker ps`. 4. Verify the SonarQube container is running.	SonarQube is running with port 9000 mapped.

```
[root@ip-172-31-7-96 /]# docker run -itpd -p 9000:9000 sonarqube
docker: invalid containerPort: d.
See 'docker run --help'.
[root@ip-172-31-7-96 /]# docker run -itd -p 9000:9000 sonarqube
40e865f7e17a7fcdc45962080f0ecee492fc9641b36decbdbcd57069291373
[root@ip-172-31-7-96 /]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
40e865f7e17a   sonarqube "/opt/sonarqube/dock..." 7 seconds ago    Up 6 seconds    0.0.0.0:9000->9000/tcp, :::9000->9000/tcp    zen_jackson
[root@ip-172-31-7-96 /]#
```

3. Generate SonarQube Token

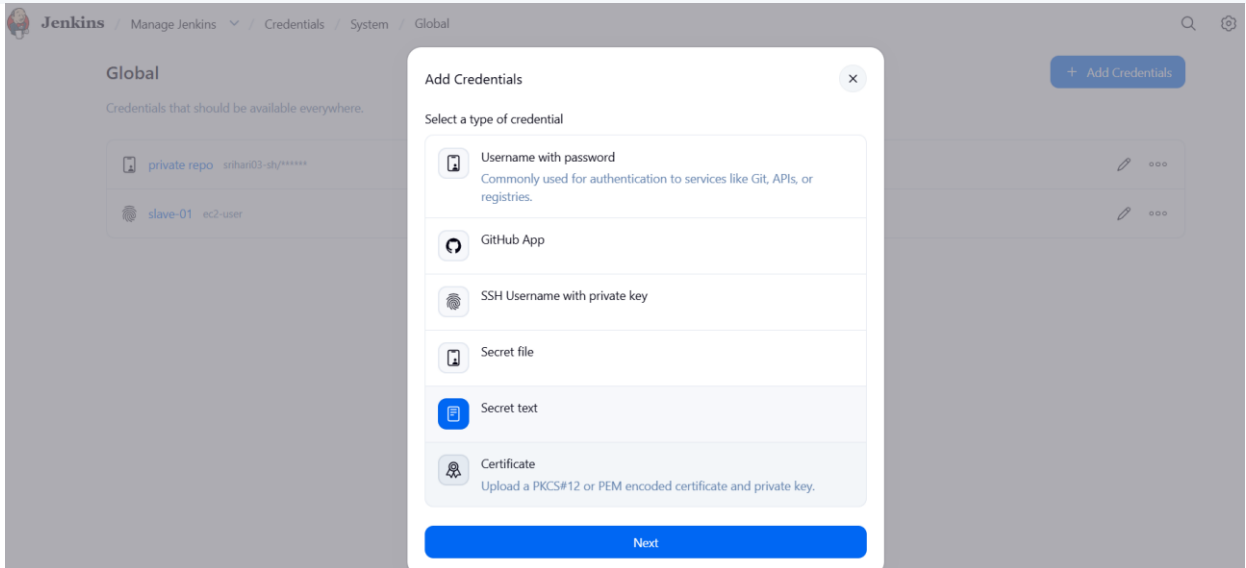
Navigation / Tabs	Action Performed	Result / Evidence
SonarQube → Administration/User Menu → Security	1. Open SonarQube. 2. Open the Security page. 3. Enter token name `sonar-token`. 4. Click Generate. 5. Copy the token when displayed.	The `sonar-token` entry is created and the token is displayed once.

The screenshot shows the SonarQube web interface. The top navigation bar includes 'Projects', 'Issues', 'Rules', 'Quality profiles', 'Quality gates', 'Administration', and 'More'. The left sidebar shows 'Administrator' and 'User' roles. The main content area is titled 'Security' and contains a message about enforcing security by using a User Token. Below this is a 'Generate Tokens' section with a form to create a new token. The form has fields for 'Name' (set to 'sonar-token'), 'Type' (set to 'Select Token Type'), and 'Expires in' (set to '30 days'). A 'Generate' button is present. Below the form, a message states: 'New token "sonar-token" has been created. Make sure you copy it now, you won't be able to see it again!'. The token value is displayed as 'squ_a8f3b26918444eaffe98f7f836a1432de97753ab'. At the bottom, there is a table with columns: Name, Type, Project, Last use, Created, and Expiration.

4. Create Jenkins Credential

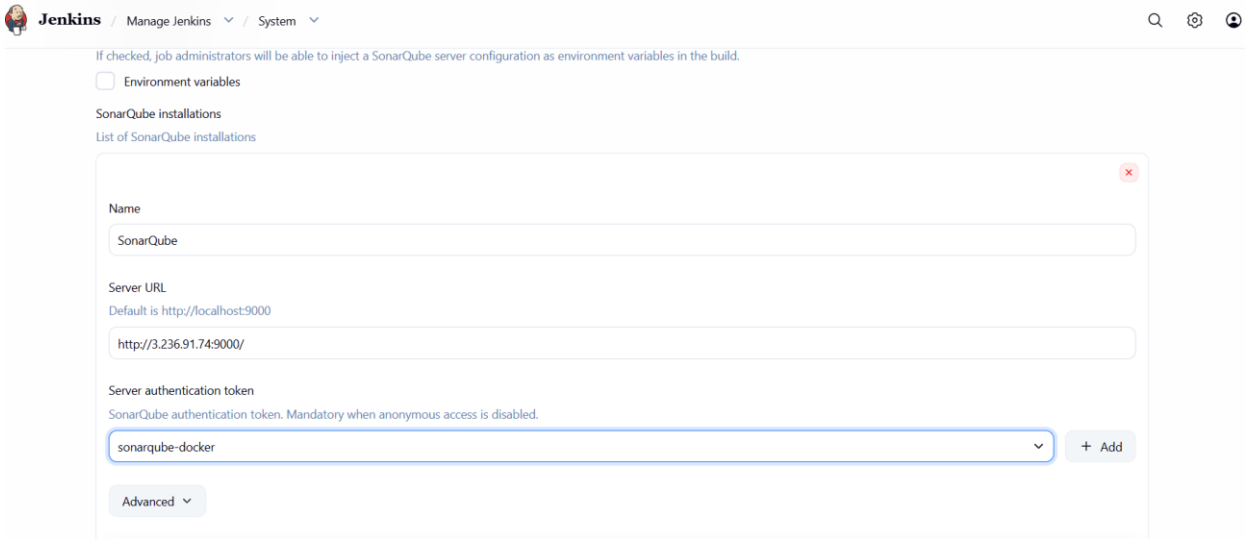
Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → Manage Jenkins → Credentials → Add Credentials	1. Open Jenkins. 2. Click Manage Jenkins. 3. Open Credentials. 4. Click Add Credentials. 5. Select the credential type for the	The Jenkins credential creation screen is configured for the SonarQube token.

- SonarQube secret.
6. Enter the secret/token.
7. Save the credential.



5. Configure SonarQube Server

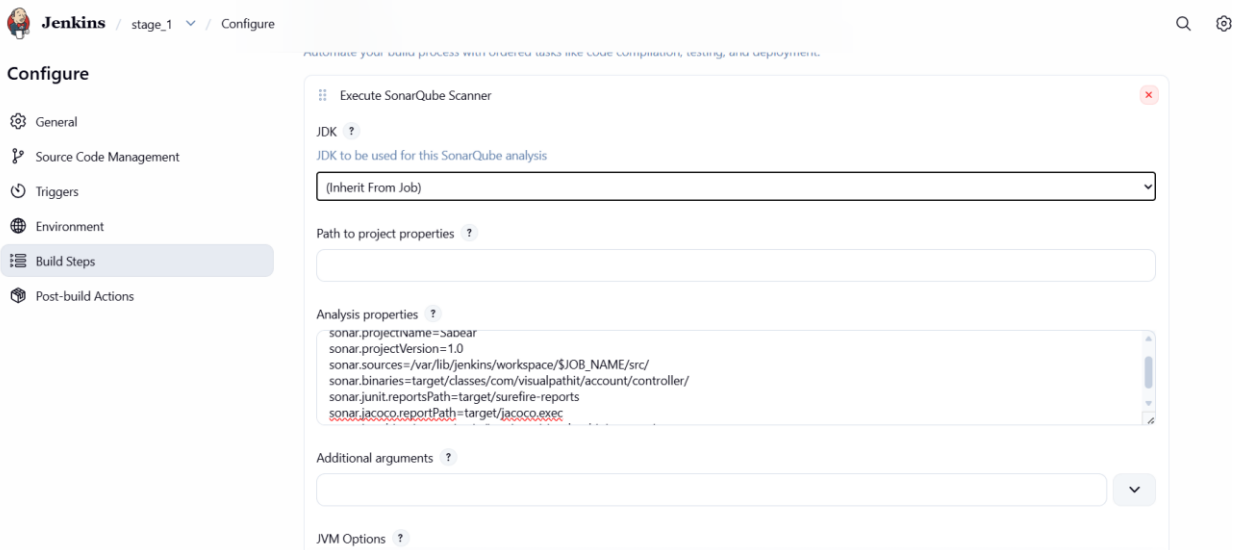
Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → Manage Jenkins → System → SonarQube servers	1. Open Manage Jenkins. 2. Click System. 3. Find SonarQube servers. 4. Add/select the SonarQube installation. 5. Enter <code>`http://3.236.91.74:9000/`</code>. 6. Select the Jenkins authentication token. 7. Save the configuration.	Jenkins has a SonarQube server configuration.



2. Task 1 — stage1 / VProfile-1

6. Configure SonarQube Scanner in stage1

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage1 → Configure → Build Steps → Execute SonarQube Scanner	1. Open the `stage1` job. 2. Click Configure. 3. Go to Build Steps. 4. Click Add build step. 5. Select Execute SonarQube Scanner. 6. Enter the analysis properties shown in the screenshot. 7. Save the job.	The SonarQube Scanner build step is configured in stage1.



7. Verify stage1 Build Result

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage1 → Build History → Build #2	1. Open `stage1`. 2. Open Build #2. 3. Check the build status. 4. Check the SonarQube link. 5. Check the SonarQube Quality Gate link.	Build #2 is stable, successful, and completed.

Jenkins / stage_1

Status

Changes

Workspace

Build Now

Delete Project

Configure

Rename

SonarQube

Builds >

Filter

Today

#3 1:16 PM

#2 1:14 PM

stage_1

SonarQube Quality Gate

Sabear

Passed

server-side processing:

Success

Permalinks

- Last build (#2), 2 min 38 sec ago
- Last stable build (#2), 2 min 38 sec ago
- Last successful build (#2), 2 min 38 sec ago
- Last completed build (#2), 2 min 38 sec ago

8. Verify SonarQube Project

Navigation / Tabs	Action Performed	Result / Evidence
SonarQube → Projects	1. Open SonarQube. 2. Click Projects. 3. Find project `Sabear` . 4. Check the quality gate. 5. Review the displayed quality metrics.	`Sabear` is visible with a Passed quality gate.

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

My Favorites All

Filters

Quality gate

Passed 1

Failed 0

Security

A ≥ 0 info issues 0

B ≥ 1 low issue 0

C ≥ 1 medium issue 1

D ≥ 1 high issue 0

E ≥ 1 blocker issue 0

Search projects

Perspective Overall Status

Sort by Name

Create Project

1 project(s)

Sabear Public

Passed

Last analysis: 1 minute ago · 1.7k Lines of Code · JSP, Java, ...

C 8

E 40

A 53

—

0.0%

2.6%

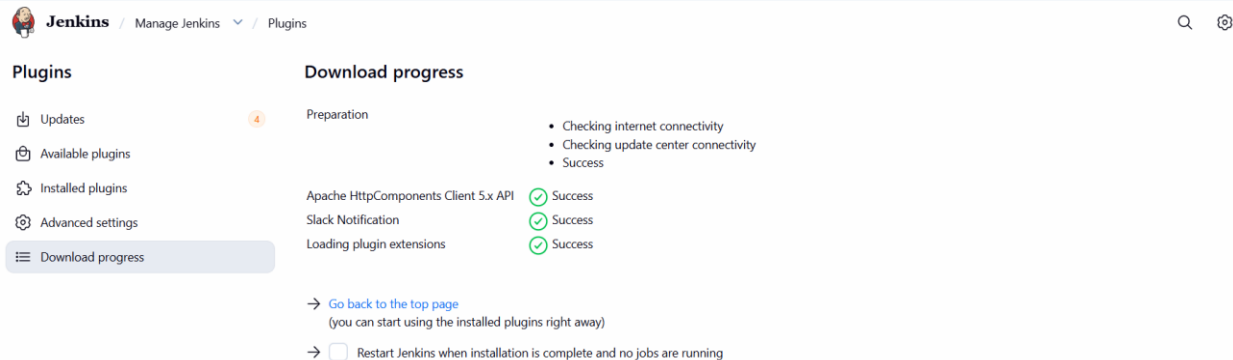
Security Reliability Maintainability Hotspots Reviewed Coverage Duplications

1 of 1 shown

3. Slack Integration

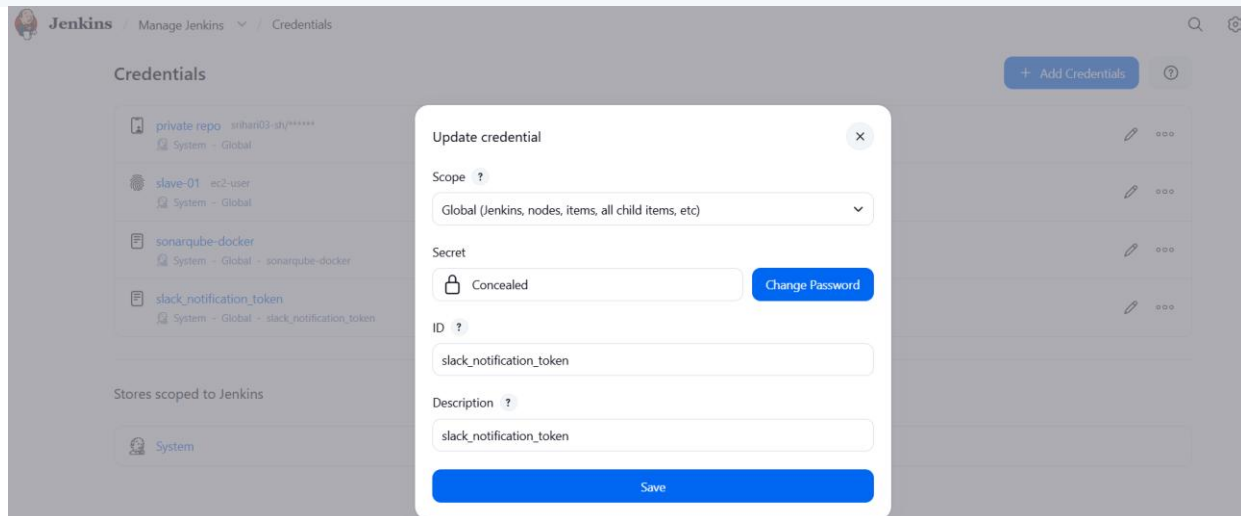
9. Install Slack Notification Plugin

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → Manage Jenkins → Plugins	1. Open Manage Jenkins. 2. Click Plugins. 3. Search for `Slack Notification`. 4. Select the Slack Notification plugin. 5. Click Install. 6. Wait for installation to complete.	The Slack Notification plugin installation succeeds.



10. Create Slack Credential

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → Manage Jenkins → Credentials → Add Credentials	1. Open Credentials. 2. Click Add Credentials. 3. Enter the Slack secret. 4. Set ID to `slack_notification_token`. 5. Set description to `slack_notification_token`. 6. Save the credential.	The Slack notification credential is stored in Jenkins.



11. Add Slack Post-build Action to stage1

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage1 → Configure → Post-build Actions → Slack Notifications	1. Open `stage1`. 2. Click Configure. 3. Go to Post-build Actions. 4. Click Add post-build action. 5. Select Slack Notifications. 6. Select the required build events. 7. Save the job.	Slack Notifications is added to stage1.

The screenshot shows the Jenkins web interface. At the top, the breadcrumb navigation is "Jenkins / stage_1 / Configure". The left sidebar contains a "Configure" section with several tabs: General, Source Code Management, Triggers, Environment, Build Steps, and Post-build Actions. The "Post-build Actions" tab is selected and highlighted. The main content area is titled "POST-BUILD ACTIONS" and includes a description: "Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs." Below this, there is a list of notification options under the "Slack Notifications" heading. The options are: "Notify Build Start" (checked), "Notify Success" (checked), "Notify Aborted" (unchecked), "Notify Not Built" (unchecked), "Notify Unstable" (unchecked), "Notify Regression" (unchecked), "Notify Every Failure" (checked), and "Notify Back To Normal" (unchecked). At the bottom of the configuration area, there is an "Advanced" dropdown menu.

12. Run stage1 and Verify Console Output

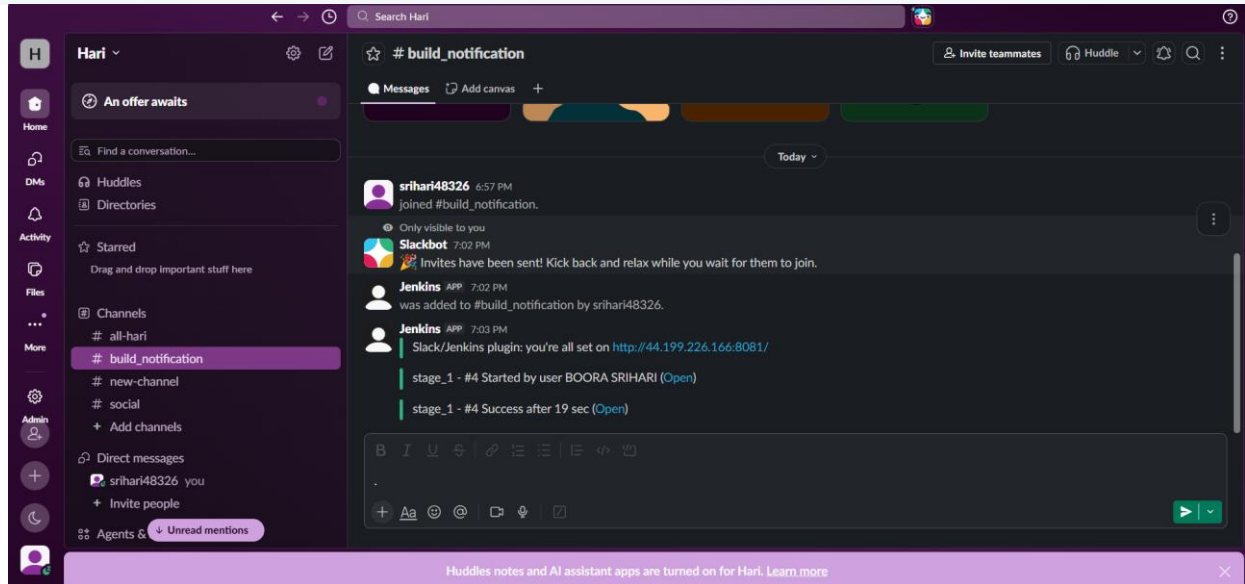
Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage1 → Build Now → Build # → Console Output	1. Open `stage1`. 2. Click Build Now. 3. Open the new build. 4. Click Console Output. 5. Check SonarQube analysis output. 6. Check the Slack notification publisher. 7. Check `Finished: SUCCESS`.	Console shows `ANALYSIS SUCCESSFUL`, `EXECUTION SUCCESS`, Slack `OnSuccessNotification`, and `Finished: SUCCESS`.



```
13:34:54.822 INFO Sensor Zero Coverage Sensor
13:34:54.846 INFO Sensor Zero Coverage Sensor (done) | time=25ms
13:34:54.846 INFO Sensor Java CPD Block Indexer
13:34:54.893 INFO Sensor Java CPD Block Indexer (done) | time=48ms
13:34:54.893 INFO ----- Gather SCA dependencies on project
13:34:54.901 INFO Dependency analysis skipped
13:34:54.916 INFO CPD Executor 8 files had no CPD blocks
13:34:54.917 INFO CPD Executor Calculating CPD for 14 files
13:34:54.942 INFO CPD Executor CPD calculation finished (done) | time=22ms
13:34:54.944 INFO SCM revision ID 'afc137167b41aaaffae82b81aee721ec883901'
13:34:55.063 INFO Analysis report generated in 116ms, dir size=664.3 kB
13:34:55.115 INFO Analysis report compressed in 50ms, zip size=146.9 kB
13:34:55.195 INFO Analysis report uploaded in 75ms
13:34:55.196 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://3.236.91.74:9000/dashboard?id=Sabear
13:34:55.198 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted anal
13:34:55.201 INFO More about the report processing at http://3.236.91.74:9000/api/ce/task?id=7bbff255-cdd3-4cef-b532-53f69efc
13:34:55.288 INFO Analysis total time: 11.340 s
13:34:55.290 INFO SonarScanner Engine completed successfully
13:34:55.315 INFO EXECUTION SUCCESS
13:34:55.316 INFO Total time: 18.550s
[Slack Notifications] found #3 as previous completed, non-aborted build
[Slack Notifications] will send OnSuccessNotification because build matches and user preferences allow it
Finished: SUCCESS
```

13. Verify stage1 Slack Message

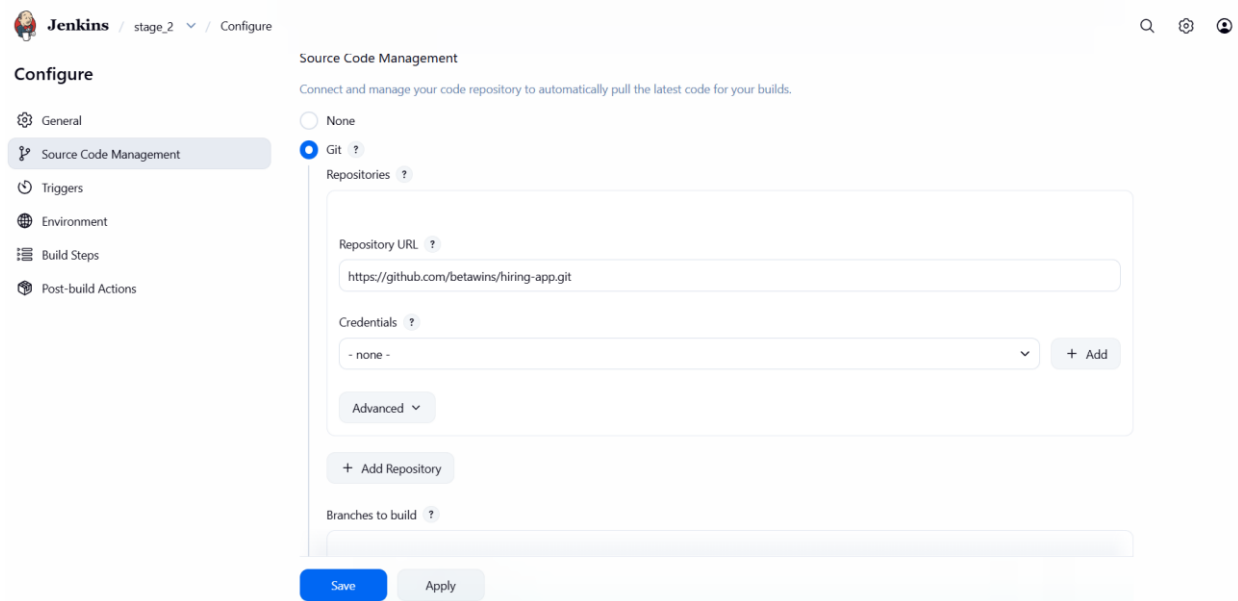
Navigation / Tabs	Action Performed	Result / Evidence
Slack → #build_notification	1. Open Slack. 2. Open `#build_notification`. 3. Find the stage1 Jenkins message. 4. Check the Started message. 5. Check the Success message.	The channel shows stage1 Started and Success notifications.



4. Task 2 — stage_2 / hiring-app

14. Configure hiring-app Git Repository

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage_2 → Configure → Source Code Management → Git	1. Open `stage_2`. 2. Click Configure. 3. Go to Source Code Management. 4. Select Git. 5. Enter `https://github.com/betawins/hiring-app.git`. 6. Save the job.	The hiring-app repository is configured.



15. Run Git Clone and Verify Checkout

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage_2 → Build Now → Build # → Console Output	1. Open `stage_2`. 2. Click Build Now. 3. Open the build. 4. Click Console Output. 5. Find Git initialization. 6. Check remote fetch. 7. Check `origin/main` checkout. 8. Check `Finished: SUCCESS`.	Jenkins successfully clones and checks out the hiring-app repository.

```

Started by user BOORA SRIHARI
Running as SYSTEM
Building on the built-in node in workspace /var/lib/jenkins/workspace/stage_2
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betawins/hiring-app.git
> git init /var/lib/jenkins/workspace/stage_2 # timeout=10
Fetching upstream changes from https://github.com/betawins/hiring-app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/hiring-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betawins/hiring-app.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9405b107fdffdf68e69fe953eaf9669784f9d6a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953eaf9669784f9d6a # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
Finished: SUCCESS

```

16. Configure SonarQube Scanner in stage_2

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage_2 → Configure → Build Steps → Execute SonarQube Scanner	1. Open `stage_2`. 2. Click Configure. 3. Go to Build Steps. 4. Click Add build step. 5. Select Execute SonarQube Scanner. 6. Set `sonar.projectKey=Test`. 7. Set `sonar.sources=.`. 8. Save the job.	The SonarQube Scanner is configured for stage_2.

Configure

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute SonarQube Scanner

JDK ?

JDK to be used for this SonarQube analysis

(Inherit From Job)

Path to project properties ?

Analysis properties ?

sonar.projectKey=Test
sonar.sources=.

17. Run SonarQube Analysis

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage_2 → Build Now → Build # → Console Output	1. Run the stage_2 build. 2. Open Console Output. 3. Find SonarQube analysis. 4. Check source analysis. 5. Check SCM processing. 6. Check report upload. 7. Check `ANALYSIS SUCCESSFUL`.	The SonarQube analysis completes successfully.

8. Check `EXECUTION SUCCESS`.

```
Jenkins stage_2 / #2
13:41:31.972 INFO Sensor JavaProjectSensor [java] (done) | time=5ms
13:41:31.973 INFO Sensor Zero Coverage Sensor
13:41:31.974 INFO Sensor Zero Coverage Sensor (done) | time=1ms
13:41:31.975 INFO ----- Gather SCA dependencies on project
13:41:31.985 INFO Dependency analysis skipped
13:41:31.986 INFO SCM Publisher SCM provider for this project is: git
13:41:31.986 INFO SCM Publisher 4 source files to be analyzed
13:41:32.283 INFO SCM Publisher 4/4 source files have been analyzed (done) | time=298ms
13:41:32.286 INFO CPD Executor 1 file had no CPD blocks
13:41:32.288 INFO CPD Executor calculating CPD for 0 files
13:41:32.289 INFO CPD Executor CPD calculation finished (done) | time=0ms
13:41:32.291 INFO SCM revision ID '9405b107fdffdf68e69fe953eaf9669784f9d6a'
13:41:32.475 INFO Analysis report generated in 174ms, dir size=290.3 kB
13:41:32.500 INFO Analysis report compressed in 23ms, zip size=34.9 kB
13:41:32.642 INFO Analysis report uploaded in 141ms
13:41:32.645 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://3.236.91.74:9000/dashboard?id=Test
13:41:32.646 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
13:41:32.648 INFO More about the report processing at http://3.236.91.74:9000/api/ce/task?id=fao0c5be-29f6-48f3-9003-7a29f33021cb
13:41:32.659 INFO Analysis total time: 4.166 s
13:41:32.661 INFO SonarScanner Engine completed successfully
13:41:32.705 INFO EXECUTION SUCCESS
13:41:32.705 INFO Total time: 11.373s
Finished: SUCCESS
```

18. Verify stage_2 Project in SonarQube

Navigation / Tabs	Action Performed	Result / Evidence
SonarQube → Projects	1. Open SonarQube. 2. Click Projects. 3. Find project `Test` . 4. Check the quality gate. 5. Review the quality indicators.	`Test` is visible with a Passed quality gate.

The way in which security, reliability, and maintainability counts and ratings are calculated has changed. [Learn more in SonarQube documentation](#)

[Projects](#)
[Issues](#)
[Rules](#)
[Quality profiles](#)
[Quality gates](#)
[Administration](#)
[More](#)

🔍
🔔
👤
🏠

My Favorites

All

Filters

Quality gate

✓ Passed 2
 ✗ Failed 0

Security

A ≥ 0 info issues 0
 B ≥ 1 low issue 1
 C ≥ 1 medium issue 1
 D ≥ 1 high issue 0
 E ≥ 1 blocker issue 0

Reliability

Create Project

2 project(s)

Perspective

Overall Status

Sort by Name

🏠 Sabear Public

✓ Passed

Last analysis: 7 minutes ago - 1.7k Lines of Code - JSP, Java, ...

C 8

E 40

A 53

—

0.0%

2.6%

Security

Reliability

Maintainability

Hotspots Reviewed

Coverage

Duplications

🏠 Test Public

✓ Passed

Last analysis: 42 seconds ago - 45 Lines of Code - XML, Docker, ...

B 2

C 3

A 0

—

—

0.0%

Security

Reliability

Maintainability

Hotspots Reviewed

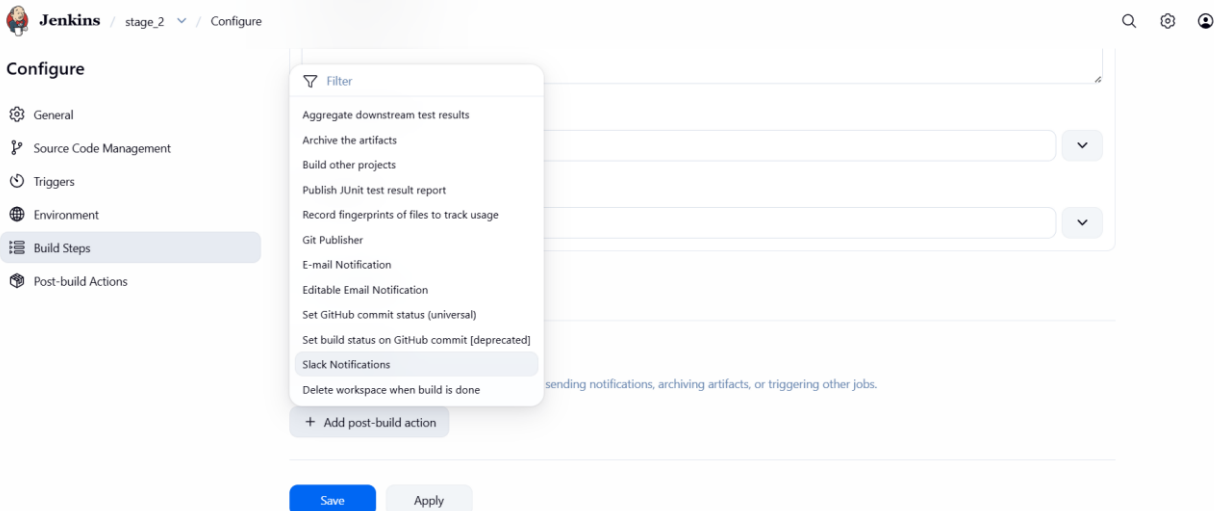
Coverage

Duplications

5. Task 2 — Slack & Final Verification

19. Add Slack Notification to stage_2

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage_2 → Configure → Post-build Actions → Add post-build action	1. Open `stage_2`. 2. Click Configure. 3. Go to Post-build Actions. 4. Click Add post-build action. 5. Select Slack Notifications.	Slack Notifications is added/selected for stage_2.



20. Configure Slack Notification Events

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage_2 → Configure → Post-build Actions → Slack Notifications	1. Open the Slack Notifications section. 2. Enable Notify Build Start if required. 3. Enable Notify Success. 4. Enable Notify Aborted if required. 5. Enable Notify Not Built if required. 6. Enable Notify Unstable if required. 7. Enable Notify Regression if required. 8. Enable Notify Every Failure if required. 9. Enable Notify Back To Normal if required. 10. Save the job.	Slack build-status notification events are configured.

Configure

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Slack Notifications

- ☒ Notify Build Start
- ☒ Notify Success
- ☐ Notify Aborted
- ☐ Notify Not Built
- ☐ Notify Unstable
- ☐ Notify Regression
- ☒ Notify Every Failure
- ☐ Notify Back To Normal

Advanced ▾

21. Run stage_2 and Verify Console Output

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage_2 → Build Now → Build # → Console Output	1. Run the stage_2 build. 2. Open Console Output. 3. Check SonarQube completion. 4. Check `ANALYSIS SUCCESSFUL`. 5. Check `EXECUTION SUCCESS`. 6. Check Slack `OnSuccessNotification`. 7. Check `Finished: SUCCESS`.	SonarQube and Slack processing complete successfully.

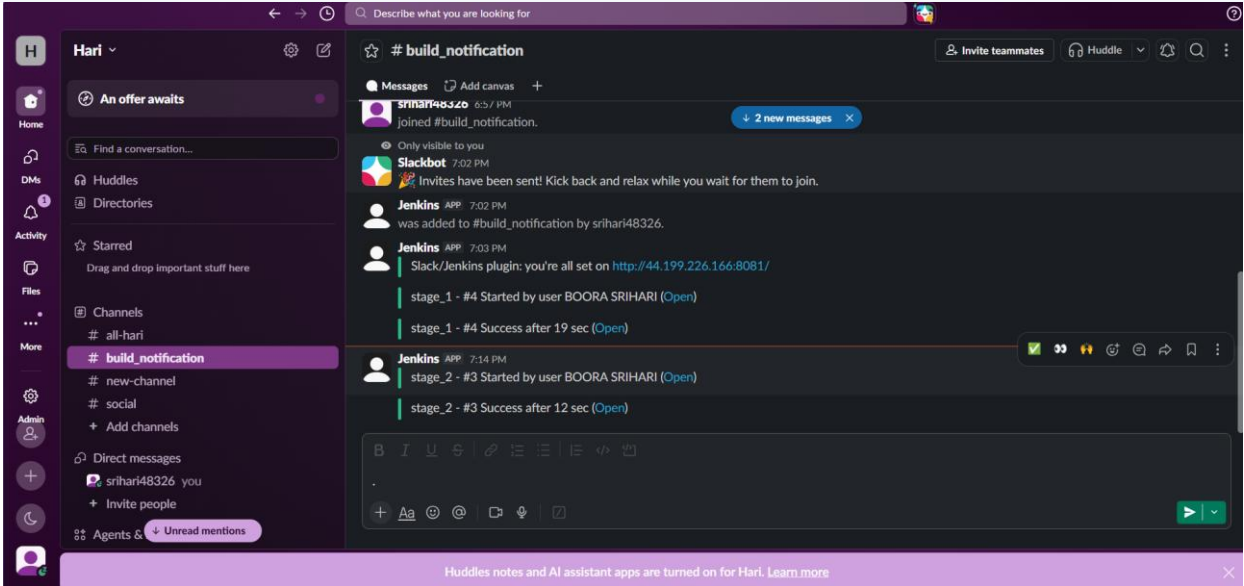
```

13:44:21.336 INFO Sensor JavaProjectSensor [java]
13:44:21.341 INFO Sensor JavaProjectSensor [java] (done) | time=4ms
13:44:21.343 INFO Sensor Zero Coverage Sensor
13:44:21.344 INFO Sensor Zero Coverage Sensor (done) | time=0ms
13:44:21.344 INFO ----- Gather SCA dependencies on project
13:44:21.349 INFO Dependency analysis skipped
13:44:21.359 INFO CPD Executor 1 file had no CPD blocks
13:44:21.359 INFO CPD Executor Calculating CPD for 0 files
13:44:21.360 INFO CPD Executor CPD calculation finished (done) | time=0ms
13:44:21.365 INFO SCM revision ID '9405b107fdffdf68e69fe953eaf9669784f9d6a'
13:44:21.493 INFO Analysis report generated in 122ms, dir size=291.0 kB
13:44:21.512 INFO Analysis report compressed in 19ms, zip size=34.3 kB
13:44:21.532 INFO Analysis report uploaded in 20ms
13:44:21.533 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://3.236.91.74:9000/dashboard?id=Test
13:44:21.535 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
13:44:21.537 INFO More about the report processing at http://3.236.91.74:9000/api/ce/task?id=475d7792-05e1-4479-b4a6-d15ff9d002e9
13:44:21.542 INFO Analysis total time: 3.802 s
13:44:21.543 INFO SonarScanner Engine completed successfully
13:44:21.562 INFO EXECUTION SUCCESS
13:44:21.563 INFO Total time: 11.116s
[Slack Notifications] found #2 as previous completed, non-aborted build
[Slack Notifications] will send OnSuccessNotification because build matches and user preferences allow it
Finished: SUCCESS

```

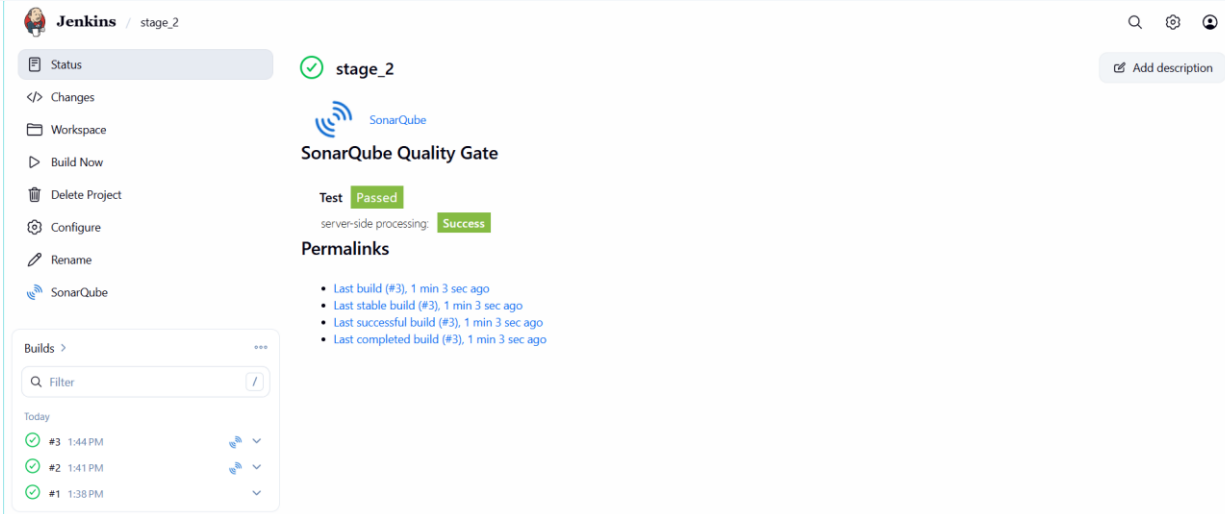
22. Verify Slack Notifications

Navigation / Tabs	Action Performed	Result / Evidence
Slack → #build_notification	1. Open Slack. 2. Open `#build_notification`. 3. Find stage1 notification. 4. Find stage2 notification. 5. Verify Started/Success messages.	Slack receives notifications from both Jenkins jobs.



23. Verify Final stage_2 Jenkins Status

Navigation / Tabs	Action Performed	Result / Evidence
Jenkins → stage_2 → Build History → Build #3	1. Open `stage_2`. 2. Open/check Build #3. 3. Check Last Build. 4. Check Last Stable Build. 5. Check Last Successful Build. 6. Check Last Completed Build. 7. Check SonarQube link. 8. Check SonarQube Quality Gate link.	Build #3 is successful and the SonarQube Quality Gate information is visible.



6. Quick Navigation Flow

Step	Navigation	Main Action
1	EC2 Terminal → Docker	Prepare SonarQube
2	SonarQube → Security	Generate token
3	Jenkins → Credentials	Store token
4	Jenkins → System	Configure SonarQube
5	Jenkins → stage1 → Configure → Build Steps	Add scanner
6	Jenkins → stage1 → Build Now → Console Output	Verify analysis
7	Jenkins → Plugins	Install Slack
8	Jenkins → stage1 → Configure → Post-build Actions	Add Slack
9	Slack → #build_notification	Verify stage1
10	Jenkins → stage_2 → Configure → SCM	Configure Git
11	Jenkins → stage_2 → Build Now → Console Output	Verify clone
12	Jenkins → stage_2 → Configure → Build Steps	Add scanner
13	SonarQube → Projects	Verify analysis
14	Jenkins → stage_2 → Configure → Post-build Actions	Add Slack
15	Slack → #build_notification	Verify stage2
16	Jenkins → stage_2 → Build History	Final verification